

- 理解链接时出现的错误。根据我们的经验，一些最令人困扰的程序错误往往都与链接器操作有关，尤其是当你试图构建大型的软件系统时。比如，链接器报告说它无法解析一个引用，这是什么意思？静态变量和全局变量的区别是什么？如果你在不同的 C 文件中定义了名字相同的两个全局变量会发生什么？静态库和动态库的区别是什么？我们在命令行上排列库的顺序有什么影响？最严重的是，为什么有些链接错误直到运行时才会出现？在第 7 章中，你将得到这些问题的答案。
- 避免安全漏洞。多年来，缓冲区溢出错误是造成大多数网络和 Internet 服务器上安全漏洞的主要原因。存在这些错误是因为很少有程序员能够理解需要限制从不受信任的源接收数据的数量和格式。学习安全编程的第一步就是理解数据和控制信息存储在程序栈上的方式会引起的后果。作为学习汇编语言的一部分，我们将在第 3 章中描述堆栈原理和缓冲区溢出错误。我们还将学习程序员、编译器和操作系统可以用来降低攻击威胁的方法。

1.4 处理器读并解释储存在内存中的指令

此刻，hello.c 源程序已经被编译系统翻译成了可执行目标文件 hello，并被存放在磁盘上。要想在 Unix 系统上运行该可执行文件，我们将它的文件名输入到称为 shell 的应用程序中：

```
linux> ./hello
hello, world
linux>
```

shell 是一个命令行解释器，它输出一个提示符，等待输入一个命令行，然后执行这个命令。如果该命令行的第一个单词不是一个内置的 shell 命令，那么 shell 就会假设这是一个可执行文件的名字，它将加载并运行这个文件。所以在此例中，shell 将加载并运行 hello 程序，然后等待程序终止。hello 程序在屏幕上输出它的消息，然后终止。shell 随后输出一个提示符，等待下一个输入的命令行。

1.4.1 系统的硬件组成

为了理解运行 hello 程序时发生了什么，我们需要了解一个典型系统的硬件组织，如图 1-4 所示。这张图是近期 Intel 系统产品族的模型，但是所有其他系统也有相同的外观和特性。现在不要担心这张图很复杂——我们将在本书分阶段对其进行详尽的介绍。

1. 总线

贯穿整个系统的是一组电子管道，称作总线，它携带信息字节并负责在各个部件间传递。通常总线被设计成传送定长的字节块，也就是字(word)。字中的字节数(即字长)是一个基本的系统参数，各个系统中都不尽相同。现在的大多数机器字长要么是 4 个字节(32 位)，要么是 8 个字节(64 位)。本书中，我们不对字长做任何固定的假设。相反，我们将在需要明确定义的上下文中具体说明一个“字”是多大。

2. I/O 设备

I/O(输入/输出)设备是系统与外部世界的联系通道。我们的示例系统包括四个 I/O 设备：作为用户输入的键盘和鼠标，作为用户输出的显示器，以及用于长期存储数据和程序的磁盘驱动器(简单地说是磁盘)。最开始，可执行程序 hello 就存放在磁盘上。

每个 I/O 设备都通过一个控制器或适配器与 I/O 总线相连。控制器和适配器之间的区

别主要在于它们的封装方式。控制器是 I/O 设备本身或者系统的主印制电路板(通常称作主板)上的芯片组。而适配器则是一块插在主板插槽上的卡。无论如何, 它们的功能都是在 I/O 总线和 I/O 设备之间传递信息。

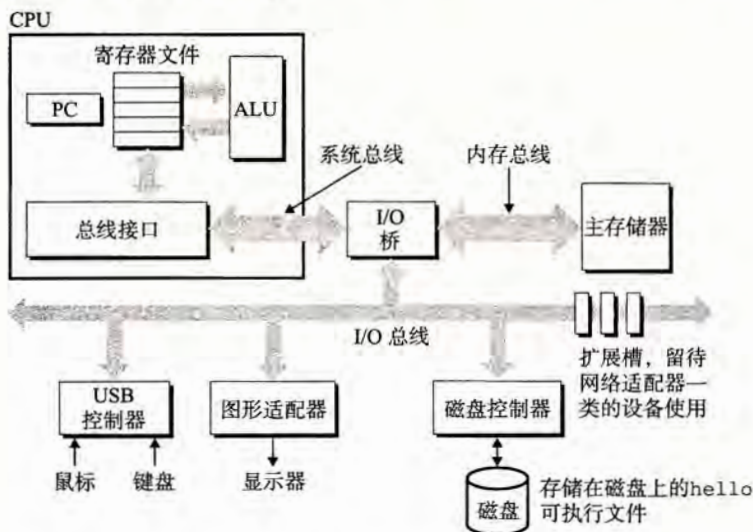


图 1-4 一个典型系统的硬件组成

CPU: 中央处理单元; ALU: 算术/逻辑单元; PC: 程序计数器; USB: 通用串行总线

第 6 章会更多地说明磁盘之类的 I/O 设备是如何工作的。在第 10 章中, 你将学习如何在应用程序中利用 Unix I/O 接口访问设备。我们将特别关注网络类设备, 不过这些技术对于其他设备来说也是通用的。

3. 主存

主存是一个临时存储设备, 在处理器执行程序时, 用来存放程序和程序处理的数据。从物理上来说, 主存是由一组动态随机存取存储器(DRAM)芯片组成的。从逻辑上来说, 存储器是一个线性的字节数组, 每个字节都有其唯一的地址(数组索引), 这些地址是从零开始的。一般来说, 组成程序的每条机器指令都由不同数量的字节构成。与 C 程序变量相对应的数据项的大小是根据类型变化的。比如, 在运行 Linux 的 x86-64 机器上, short 类型的数据需要 2 个字节, int 和 float 类型需要 4 个字节, 而 long 和 double 类型需要 8 个字节。

第 6 章将具体介绍存储器技术, 比如 DRAM 芯片是如何工作的, 它们又是如何组合起来构成主存的。

4. 处理器

中央处理单元(CPU), 简称处理器, 是解释(或执行)存储在主存中指令的引擎。处理器的核心是一个大小为一个字的存储设备(或寄存器), 称为程序计数器(PC)。在任何时刻, PC 都指向主存中的某条机器语言指令(即含有该条指令的地址)。^①

从系统通电开始, 直到系统断电, 处理器一直在不断地执行程序计数器指向的指令, 再更新程序计数器, 使其指向下一条指令。处理器看上去是按照一个非常简单的指令执行模型来操作的, 这个模型是由指令集架构决定的。在这个模型中, 指令按照严格的顺序执行, 而执行一条指令包含执行一系列的步骤。处理器从程序计数器指向的内存处读取指

^① PC 也普遍地被用来作为“个人计算机”的缩写。然而, 两者之间的区别应该可以很清楚地从上下文中看出来。

令，解释指令中的位，执行该指令指示的简单操作，然后更新 PC，使其指向下一条指令，而这条指令并不一定和在内存中刚刚执行的指令相邻。

这样的简单操作并不多，它们围绕着主存、寄存器文件(register file)和算术/逻辑单元(ALU)进行。寄存器文件是一个小的存储设备，由一些单个字长的寄存器组成，每个寄存器都有唯一的名字。ALU 计算新的数据和地址值。下面是一些简单操作的例子，CPU 在指令的要求下可能会执行这些操作。

- 加载：从主存复制一个字节或者一个字到寄存器，以覆盖寄存器原来的内容。
- 存储：从寄存器复制一个字节或者一个字到主存的某个位置，以覆盖这个位置上原来的内容。
- 操作：把两个寄存器的内容复制到 ALU，ALU 对这两个字做算术运算，并将结果存放到一个寄存器中，以覆盖该寄存器中原来的内容。
- 跳转：从指令本身中抽取一个字，并将这个字复制到程序计数器(PC)中，以覆盖 PC 中原来的值。

处理器看上去是它的指令集架构的简单实现，但是实际上现代处理器使用了非常复杂的机制来加速程序的执行。因此，我们将处理器的指令集架构和处理器的微体系结构区分开来：指令集架构描述的是每条机器代码指令的效果；而微体系结构描述的是处理器实际上是如何实现的。在第3章研究机器代码时，我们考虑的是机器的指令集架构所提供的抽象性。第4章将更详细地介绍处理器实际上是如何实现的。第5章用一个模型说明现代处理器是如何工作的，从而能预测和优化机器语言程序的性能。

1.4.2 运行 hello 程序

前面简单描述了系统的硬件组成和操作，现在开始介绍当我们运行示例程序时到底发生了些什么。在这里必须省略很多细节，稍后会做补充，但是现在我们将很满意于这种整体上的描述。

初始时，shell 程序执行它的指令，等待我们输入一个命令。当我们在键盘上输入字符串“./hello”后，shell 程序将字符逐一读入寄存器，再把它存放到内存中，如图 1-5 所示。

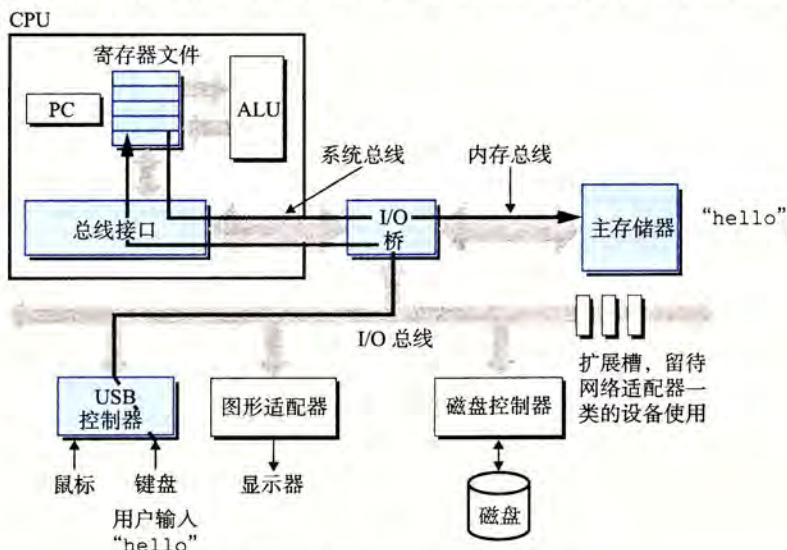


图 1-5 从键盘上读取 hello 命令

当我们在键盘上敲回车键时，shell 程序就知道我们已经结束了命令的输入。然后 shell 执行一系列指令来加载可执行的 hello 文件，这些指令将 hello 目标文件中的代码和数据从磁盘复制到主存。数据包括最终会被输出的字符串“hello, world\n”。

利用直接存储器存取(DMA，将在第6章中讨论)技术，数据可以不通过处理器而直接从磁盘到达主存。这个步骤如图1-6所示。

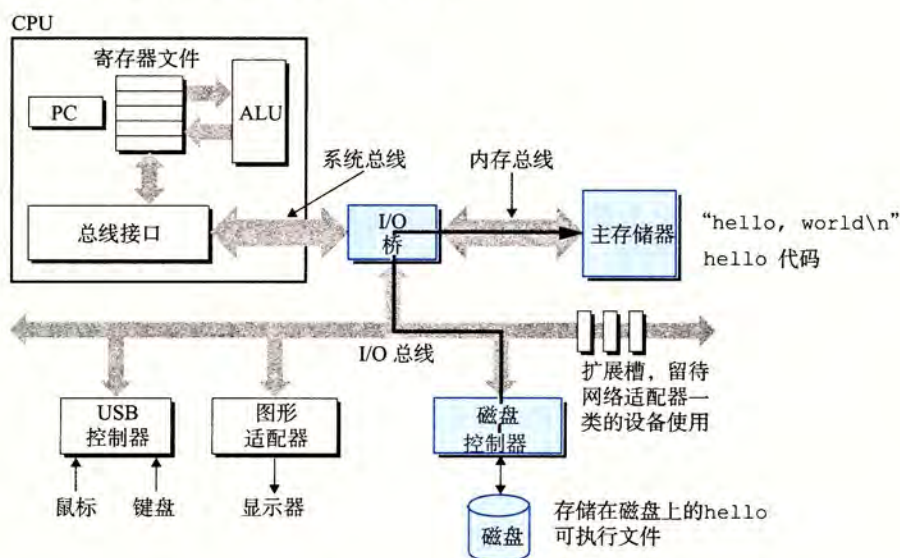


图 1-6 从磁盘加载可执行文件到主存

一旦目标文件 hello 中的代码和数据被加载到主存，处理器就开始执行 hello 程序的 main 程序中的机器语言指令。这些指令将“hello, world\n”字符串中的字节从主存复制到寄存器文件，再从寄存器文件中复制到显示设备，最终显示在屏幕上。这个步骤如图1-7所示。

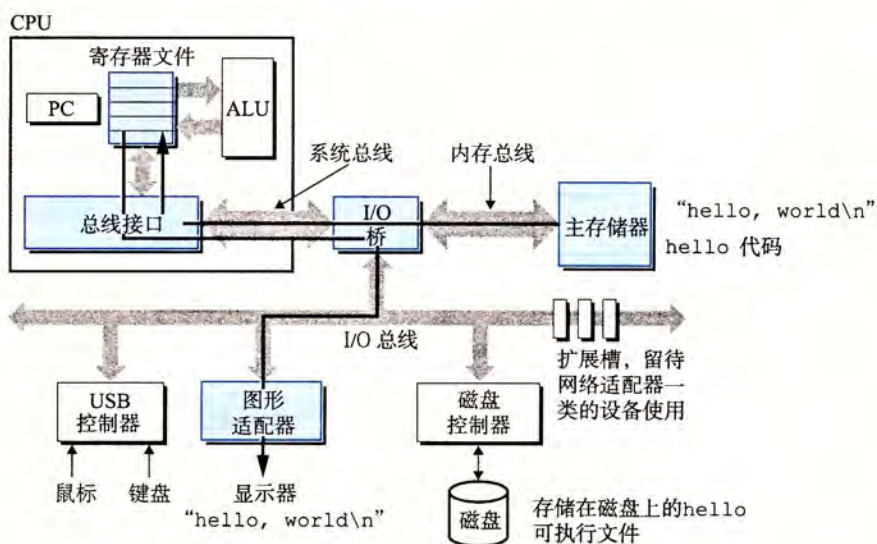


图 1-7 将输出字符串从存储器写到显示器

1.5 高速缓存至关重要

这个简单的示例揭示了一个重要的问题，即系统花费了大量的时间把信息从一个地方搬到另一个地方。hello 程序的机器指令最初是存放在磁盘上，当程序加载时，它们被复制到主存；当处理器运行程序时，指令又从主存复制到处理器。相似地，数据串“hello, world/n”开始时在磁盘上，然后被复制到主存，最后从主存上复制到显示设备。从程序员的角度来看，这些复制就是开销，减慢了程序“真正”的工作。因此，系统设计者的一个主要目标就是使这些复制操作尽可能快地完成。

根据机械原理，较大的存储设备要比较小的存储设备运行得慢，而快速设备的造价远高于同类的低速设备。比如说，一个典型系统上的磁盘驱动器可能比主存大 1000 倍，但是对于处理器而言，从磁盘驱动器上读取一个字的时间开销要比从主存中读取的开销大 1000 万倍。

类似地，一个典型的寄存器文件只存储几百字节的信息，而主存里可存放几十亿字节。然而，处理器从寄存器文件中读数据比从主存中读取几乎要快 100 倍。更麻烦的是，随着这些年半导体技术的进步，这种处理器与主存之间的差距还在持续增大。加快处理器的运行速度比加快主存的运行速度要容易和便宜得多。

针对这种处理器与主存之间的差异，系统设计者采用了更小更快的存储设备，称为高速缓存存储器(cache memory，简称为 cache 或高速缓存)，作为暂时的集结区域，存放处理器近期可能会需要的信息。图 1-8 展示了一个典型系统中的高速缓存存储器。位于处理器芯片上的 L1 高速缓存的容量可以达到数万字节，访问速度几乎和访问寄存器文件一样快。一个容量为数十万到数百万字节的更大的 L2 高速缓存通过一条特殊的总线连接到处理器。进程访问 L2 高速缓存的时间要比访问 L1 高速缓存的时间长 5 倍，但是这仍然比访问主存的时间快 5~10 倍。L1 和 L2 高速缓存是用一种叫做静态随机访问存储器(SRAM)的硬件技术实现的。比较新的、处理能力更强大的系统甚至有三级高速缓存：L1、L2 和 L3。系统可以获得一个很大的存储器，同时访问速度也很快，原因是利用了高速缓存的局部性原理，即程序具有访问局部区域里的数据和代码的趋势。通过让高速缓存里存放可能经常访问的数据，大部分的内存操作都能在快速的高速缓存中完成。

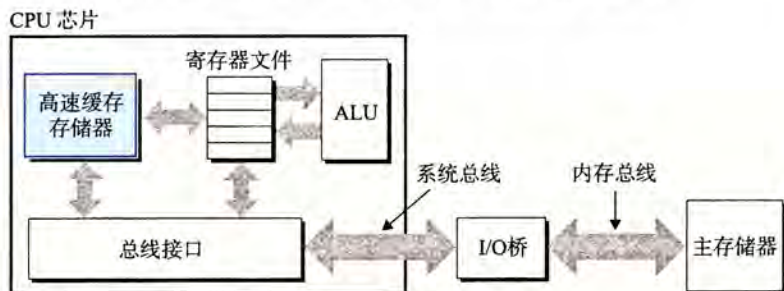


图 1-8 高速缓存存储器

本书得出的重要结论之一就是，意识到高速缓存存储器存在的应用程序员能够利用高速缓存将程序的性能提高一个数量级。你将在第 6 章里学习这些重要的设备以及如何利用它们。

1.6 存储设备形成层次结构

在处理器和一个较大较慢的设备（例如主存）之间插入一个更小更快的存储设备（例如高速缓存）的想法已经成为一个普遍的观念。实际上，每个计算机系统存储设备都被